

Step 1 — Security Reviewer Chain — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

This project is deliberately lighter on hand-holding than Projects 1-5 — these hints are nudges toward the right direction, not step-by-step answers. Try genuinely hard before reading Hint 2. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — What "Step 1" means here](#)
- [Hint 2 — A nudge on the shape, not the code](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What "Step 1" means here

Basic Version

You've already built four "Step 1"s across Projects 1-4. Every one of them had the same shape: the smallest piece of core logic, proven alone, with no agent behavior, no tools, no team around it. This project's Step 1 title even tells you directly — "One Reviewer, Working as a Plain Chain." Ask yourself what that phrase ruled out before you start designing anything.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Project 4's Reviewer (Step 2) already gave you a template worth reusing directly: a chain that takes some text in, and returns a structured judgment out — not free text you'd have to re-parse. Think about what the equivalent structured output should look like for a *security* reviewer specifically. A plain approve/reject flag isn't quite it here — a code reviewer usually needs to say *what* it found and *how severe* each thing is, not just pass/fail on the whole diff.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Before you write a single prompt, look at a real `git diff` output and notice something about its format: lines starting with `+` were added, lines starting with `-` were removed, and everything else is unchanged context shown for readability. This project's own problems table warns about a Security reviewer flagging something that "didn't actually change." Think hard about what that implies for how you feed a diff into your prompt — should the model see the whole file, or only the lines that actually changed? What would go wrong if you got that decision wrong in either direction (too little context to judge correctly, vs. flagging pre-existing issues that aren't this PR's fault)?

Difference between Basic, Intermediate, and Advanced: Basic points you at the *scope* of this step (smallest piece, no agent yet). Intermediate points you at the *shape* of a good output (structured findings, not a pass/fail flag). Advanced points you at a real correctness trap in the *input* (diff format) that's easy to get wrong on the first pass and easy to miss if you don't look at a real diff before designing your prompt.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — A nudge on the shape, not the code

Basic Version

You need: something that represents one finding (what's wrong, how bad it is), something that represents the whole review (a list of findings), a prompt that asks the model to look specifically for security problems (hardcoded secrets, obviously unsafe patterns), and 2-3 made-up diffs to test against — at least one with an obvious planted problem, and at least one clean diff that should come back with no findings.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Revisit how Project 4's Reviewer got a typed verdict back reliably (`.with_structured_output(...)` on a Pydantic model) — the same mechanism applies here, just with a `Finding` model (something like `severity`, `message`, maybe `line_hint`) and a list of them, instead of one `approved / feedback` pair. Think about what severities are actually useful to distinguish (does "critical vs. minor" matter more than a numeric score here?), and write that into the model's field descriptions the way Project 4's `ReviewVerdict` did.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Try building your clean test diff so it contains an *unchanged* line that looks like a secret (say, a pre-existing hardcoded key sitting a few lines above the actual change), specifically to see if your first prompt attempt flags it anyway. If it does, that's the exact bug this project's problems table names — and it's a much better lesson learned from your own test catching it than from being told about it up front. Fix your prompt (and how you build the diff text you hand it) until that test genuinely passes.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces you'll need. Intermediate points you back at a mechanism you've already used successfully in Project 4, applied to a slightly richer shape. Advanced asks you to go find the "flags something unchanged" bug yourself, with a test built specifically to expose it, rather than handing you the fix directly.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)